CO1
ASSESSMENT TOOL 1
Pattern Recognition and Text Processing

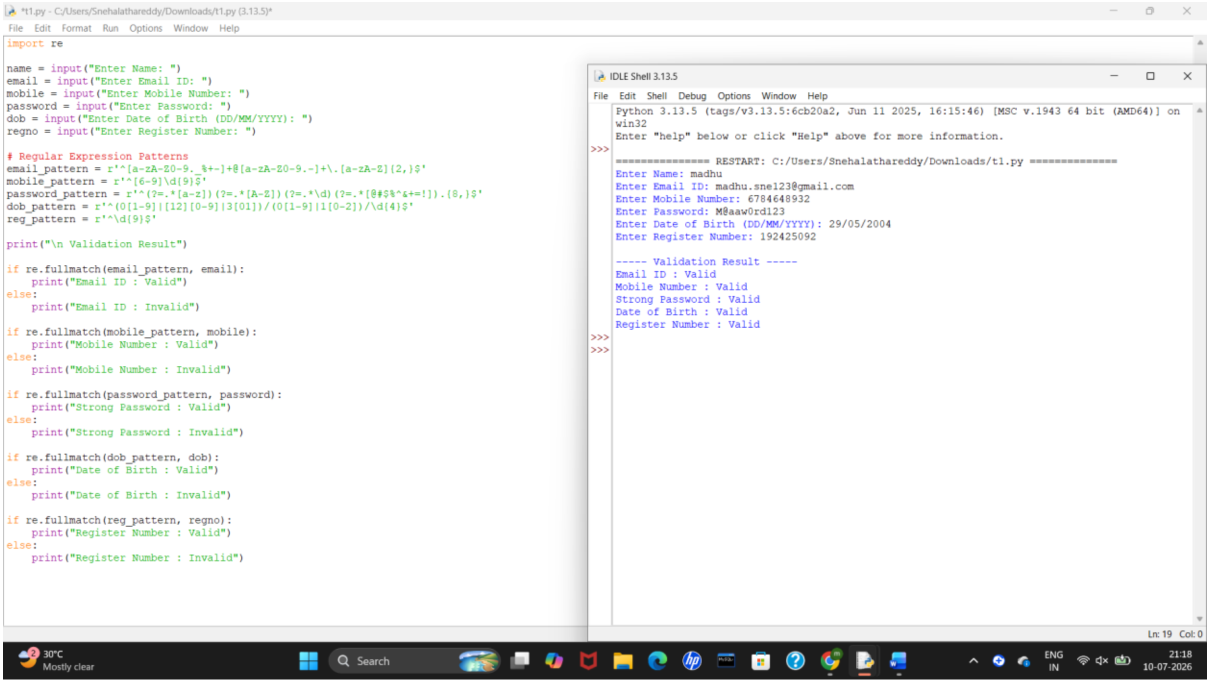
Section 1: Regular Expression Pattern Generation
Student Details:
Name: John Doe
Email: john.doe123@gmail.com
Mobile: +91-9876543210
Password: P@ssw0rd123
Date of Birth: 15/08/2004
Register Number: 23AIML1056
Department: Artificial Intelligence and Machine Learning
For the given student details formulate Regex patterns for the following:
Email ID
Mobile Number
Strong Password
Date of Birth (DD/MM/YYYY)
Register Number

PYTHON CODE:

```
import re
name = input("Enter Name: ")
email = input("Enter Email ID: ")
mobile = input("Enter Mobile Number: ")
password = input("Enter Password: ")
dob = input("Enter Date of Birth (DD/MM/YYYY): ")
regno = input("Enter Register Number: ")

email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
mobile_pattern = r'^[6-9]\d{9}$'
password_pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#$%^&+=!]).{8,}$'
dob_pattern = r'^(0[1-9]||[12][0-9]||3[01])/(0[1-9]||1[0-2])/\d{4}$'
reg_pattern = r'^\d{2}[A-Z]{4}\d{4}$'

print("\n----- Validation Result -----")
if re.fullmatch(email_pattern, email):
   print("Email ID : Valid")
else:
   print("Email ID : Invalid")

if re.fullmatch(mobile_pattern, mobile):
   print("Mobile Number : Valid")
else:
   print("Mobile Number : Invalid")

if re.fullmatch(password_pattern, password):
   print("Strong Password : Valid")
else:
   print("Strong Password : Invalid")
```

```
if re.fullmatch(dob_pattern, dob):
    print("Date of Birth : Valid")
else:
    print("Date of Birth : Invalid")

if re.fullmatch(reg_pattern, regno):
    print("Register Number : Valid")
else:
    print("Register Number : Invalid")
```

output:

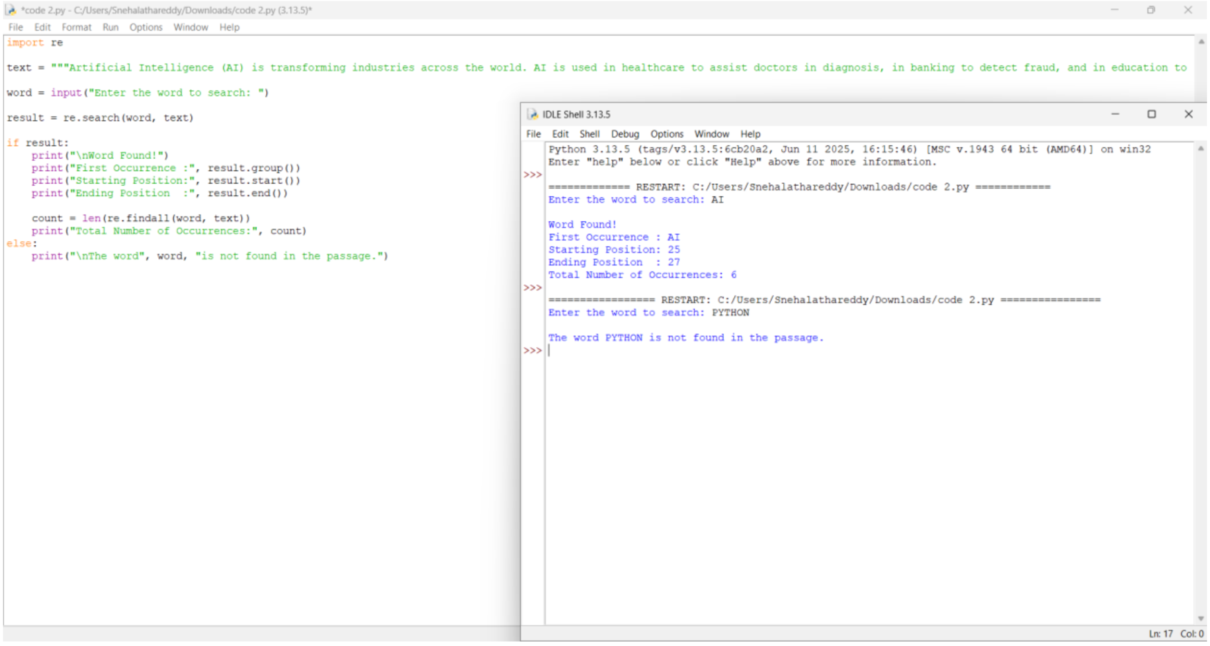


Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.
Write a Python program using the re module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using re.search().
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage.

Print an appropriate message if the word is not found.



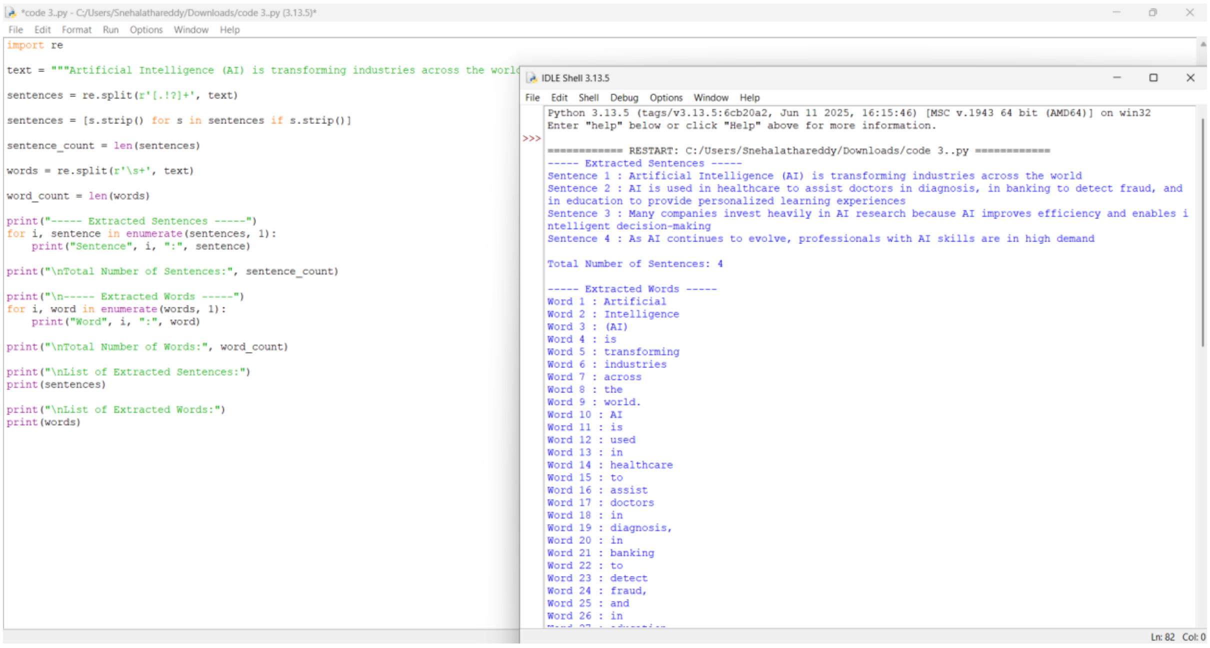
Section 3: Split the documentation into sentences and words

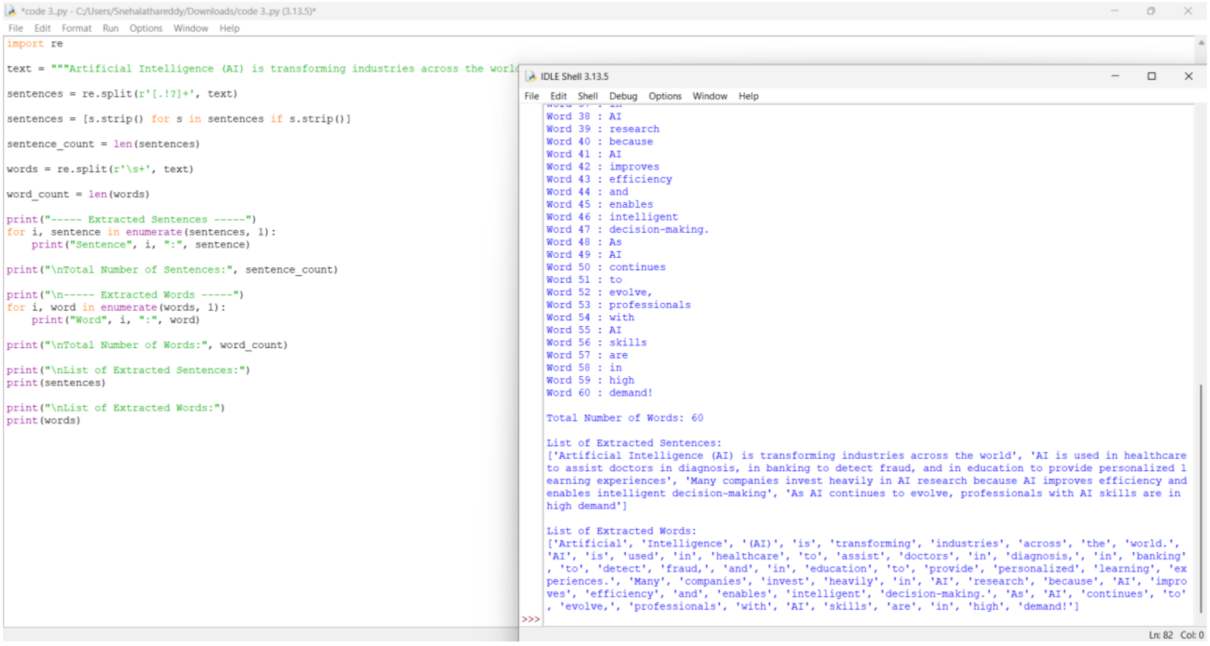
Refer the previous passage and Write a Python program using the re.split() method to perform the following operations:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count and display the total number of sentences.
- Split the passage into words by considering one or more whitespace characters as delimiters.
- Count and display the total number of words.

Display the list of extracted sentences and words.

OUTPUT:





```
*code 3..py - C:/Users/Snehalathareddy/Downloads/code 3..py (3.13.5)*
File Edit Format Run Options Window Help
import re

text = """Artificial Intelligence (AI) is transforming industries across the worl

sentences = re.split(r'[.!?]+', text)

sentences = [s.strip() for s in sentences if s.strip()]

sentence_count = len(sentences)

words = re.split(r'\s+', text)

word_count = len(words)

print("----- Extracted Sentences -----")
for i, sentence in enumerate(sentences, 1):
    print("Sentence", i, ":", sentence)

print("\nTotal Number of Sentences:", sentence_count)

print("\n----- Extracted Words -----")
for i, word in enumerate(words, 1):
    print("Word", i, ":", word)

print("\nTotal Number of Words:", word_count)

print("\nList of Extracted Sentences:")
print(sentences)

print("\nList of Extracted Words:")
print(words)
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Word 38 : AI
Word 39 : research
Word 40 : because
Word 41 : AI
Word 42 : improves
Word 43 : efficiency
Word 44 : and
Word 45 : enables
Word 46 : intelligent
Word 47 : decision-making.
Word 48 : As
Word 49 : AI
Word 50 : continues
Word 51 : to
Word 52 : evolve,
Word 53 : professionals
Word 54 : with
Word 55 : AI
Word 56 : skills
Word 57 : are
Word 58 : in
Word 59 : high
Word 60 : demand!

Total Number of Words: 60

List of Extracted Sentences:
['Artificial Intelligence (AI) is transforming industries across the world', 'AI is used in healthcare
to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized l
earning experiences', 'Many companies invest heavily in AI research because AI improves efficiency and
enables intelligent decision-making', 'As AI continues to evolve, professionals with AI skills are in
high demand']

List of Extracted Words:
['Artificial', 'Intelligence', '(AI)', 'is', 'transforming', 'industries', 'across', 'the', 'world.',
'AI', 'is', 'used', 'in', 'healthcare', 'to', 'assist', 'doctors', 'in', 'diagnosis,', 'in', 'banking'
, 'to', 'detect', 'fraud,', 'and', 'in', 'education', 'to', 'provide', 'personalized', 'learning', 'ex
periences.', 'Many', 'companies', 'invest', 'heavily', 'in', 'AI', 'research', 'because', 'AI', 'impro
ves', 'efficiency', 'and', 'enables', 'intelligent', 'decision-making.', 'As', 'AI', 'continues', 'to'
, 'evolve,', 'professionals', 'with', 'AI', 'skills', 'are', 'in', 'high', 'demand!']
>>>
Ln: 82 Col: 0
```

Section 4: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the re module to validate the following inputs.

Validation Criteria:

Email ID

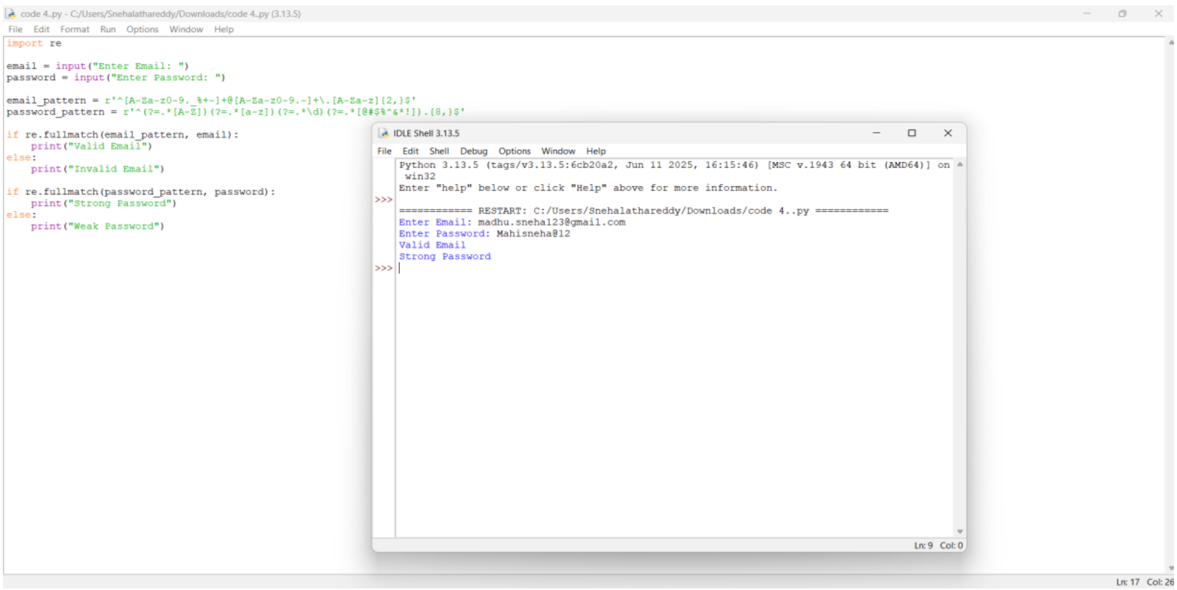
- Must begin with one or more letters or digits.
- May contain ., _, %, +, or - before the @ symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as .com, .org, .edu, or .in.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from @, #, $, %, &, !, or *.

Must not contain any whitespace characters.

OUTPUT:



```
code 4..py - C:/Users/Snehalathareddy/Downloads/code 4..py (3.13.5)
File Edit Format Run Options Window Help
import re

email = input("Enter Email: ")
password = input("Enter Password: ")

email_pattern = r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%^&*!]).{8,}$'

if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")
Ln: 17 Col: 26
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on
 win32
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:/Users/Snehalathareddy/Downloads/code 4..py ============
Enter Email: madhu.sneha123@gmail.com
Enter Password: Mahisneha@12
Valid Email
Strong Password
>>>
Ln: 9 Col: 0
```

Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the re.compile() method to compile the pattern once and reuse it to validate all the email IDs.

PYTHON CODE:

```
import re

email_pattern = re.compile(r'^[A-Za-z0-9][A-Za-z0-9._%+-]*@[A-Za-z0-9-]+\.(com|org|edu|in)$')

n = int(input("Enter the number of Email IDs: "))

print("\nEnter the Email IDs:")

for i in range(n):
    email = input("Email {}: ".format(i + 1))

    if email_pattern.fullmatch(email):
        print(email, "-> Valid Email ID")
    else:
        print(email, "-> Invalid Email ID")
```

OUTPUT: